6 Conclusions and future work

This project developed a finite element framework for partial differential equations posed on metric graphs and applied it to a problem of current interest: the Fisher–Kolmogorov model of neurodegeneration on the brain connectome. The library is built around a compact core consisting of a graph topology with metric connections, a piecewise linear discretisation along each connection, and a common workflow for coefficient evaluation, assembly, solution and export. The benchmarks of Section 3 validated this framework on problems with known solutions: stationary and time-dependent problems on the star graph, showing the expected first-order accuracy of Backward Euler and second-order accuracy of Crank–Nicolson in time, eigenvalue problems on the star, graphene-like and binary tree graphs against the reference spectra of [6], and the comparison between the metric-graph operator and the combinatorial graph Laplacian.

The study of Section 4 used the library as a laboratory for the Fisher–Kolmogorov equation on the 83-region connectome of [9]. Three findings stand out. First, at one element per connection the metric-graph formulation reproduces the behaviour of the nodal network models while remaining computationally inexpensive, and refinement inside the connections remains available whenever sharper fronts require it. Second, the balance between reaction and transport, summarised by the Damköhler number, governs both the physical regime and the numerical robustness of the model, and the lumped mass matrix is the appropriate choice in the reaction-dominated regime. Third, connectivity alone orders the four lobes only partially: with the regionally heterogeneous reaction rates of [8] transferred to our connectome, the lobes cross the staging thresholds in the clinical order, suggesting a division of roles between the transport pathways and the regional reaction rates.

The performance analysis of Section 5 closed the loop between the mathematics and the implementation. The unexpected cost of the sequential baseline was traced to the fill-in generated by the default matrix ordering. The natural numbering of the unknowns, with connection interiors first and graph vertices last, exposes a bordered structure that can be exploited through static condensation. By eliminating the interior unknowns locally, the global problem is reduced to the 83 original graph vertices, while the remaining work becomes naturally parallel over the connections. On a four-core laptop, combining this reduction with parallel execution lowers the per-step cost of the finest connectome discretisation by a factor of about thirty with respect to the best full-system factorisation. Each intermediate optimisation was verified against the sequential solver up to round-off error.

Several directions remain open.

- The MPI version was measured only on a single shared-memory node, where the OpenMP implementation is preferable. Since the main purpose of MPI is to enable distributed-memory execution, a multi-node scalability study is the natural next step.
- Static condensation currently accelerates the semi-implicit scheme. Extending it to the fully implicit scheme means applying the same reduction inside every Newton iteration, whose matrices share the sparsity structure of the semi-implicit one.
- A per-step cost of about two milliseconds at 143593 unknowns makes many-run studies practical; parameter calibration and the uncertainty quantification of [8] are the settings in which the speedup would be most valuable.
- The data pipeline already reconstructs the fine graph with 1015 nodes from which the 83 regions are aggregated; repeating the study at the finer parcellation would test the robustness of the staging results to the aggregation.

- Richer disease models from the literature, with several protein species or patient-specific parameters, can be hosted on the same discretisation layer without changes to its structure.

The quantitative results reported throughout this work are backed by stored and regenerable benchmark records, as described in the next section. Together with the validation of each optimisation against the reference solver, this reproducibility is an important outcome of the project itself.

7 C++ implementation

7.1 Repository reference

The project is delivered as a single repository whose main entries separate the library, the experiments and their records:

```
FEMG/            the library: headers and sources of the problem classes
test_problem/    one driver per experiment family (parabolic, spectral,
                 fisher_kolmogorov, performance)
benchmarks/      one numbered record per experiment of the report, each
                 with a README, the commands used and the stored results
scripts/         benchmark runners, figure generation and verification
data/            graph files and the connectome preparation pipeline
verification/    internal correctness checks of the solver
report/          this document
```

The records are the organising principle: every figure and every number of the report is produced from files stored under `benchmarks/`, and each record documents how to regenerate them.

7.2 Dependencies

The library is written in C++17 and built with CMake (3.16 or later). It depends on Eigen 3.4 for sparse and dense linear algebra and on the Boost Graph Library (1.76, the version provided by the mk toolchain of the course) for the graph topology. OpenMP and MPI are optional: the performance executables of Section 5 are added to the build only when CMake finds them and the library itself requires neither. Post-processing uses Python 3 with NumPy, Matplotlib and pandas. The default build type is Release with `-O2`.

7.3 Computational approach to differential problems

All problems share the abstract base class `quantum_graph_problem`. The topology is a Boost undirected adjacency list whose vertex property carries the coordinates used for visualisation and whose edge property carries the metric data of the connection, its length and its number of finite element cells. `init()` reads the graph file and builds the numbering of the unknowns, interior nodes of each connection first and the original vertices last, the layout on which Sections 5.2 and 5.3 rely. The base class stores the shared discrete objects, the discrete Hamiltonian H, the mass matrix M and the solution vector, and prescribes the workflow as four virtual steps: `set_coefficients()`, `assembly()`, `solve()` and `sol_export()`.

Three concrete classes derive from it. `parabolic_problem` implements the θ-method for the heat equation with Dirichlet conditions on selected vertices and source and initial data supplied as callables; as stated in Section 3.1, the elliptic benchmarks are stationary solutions computed through this parabolic pipeline. `spectral_problem` solves the generalised eigenvalue problem $Hx = \lambda Mx$ for a requested number of modes and also exposes the eigenvalues of the combinatorial Laplacian for the comparison of Section 3.4.3. `fisher_kolmogorov_problem` implements the two time schemes of Table 5, Backward Euler with a damped and safeguarded

Newton iteration and the semi-implicit scheme of [8], with one diffusion coefficient per connection, one reaction rate per region and optional mass lumping. The condensed solver of Section 5 is deliberately kept outside the library, as a header-only engine under `test_problem/performance/` that replays the semi-implicit step without modifying the validated classes.

7.4 Input data

Graphs are plain text files. The header carries the number of vertices and connections; an optional block with two coordinates per vertex follows, and each connection is described by its two endpoint indices, its length and its number of cells. The same reader accepts both the coordinate-free and the coordinate-carrying variant. The `data/` directory provides the analytical geometries of Section 3, the interval, the four-pointed star, the graphene-like graph and the binary tree, at several resolutions. The connectome enters through a documented preparation pipeline: the public files of the Budapest Reference Connectome [10] are reduced to the 83-region connectome of [9] by the scripts under `scripts/`, which also emit the per-connection weights and the refined meshes, from one to 128 elements per connection, used in Sections 4 and 5. Every derived file is regenerable from the raw download.

7.5 Setting coefficients

Physical parameters travel through constructors and setters. The parabolic constructor takes the diffusion coefficient, the final time, the time step and θ; sources, initial data and vertex Dirichlet data are function objects evaluated on the metric coordinate of each connection. The Fisher–Kolmogorov class accepts one diffusion coefficient per connection, in Section 4 the normalised fibre weights, and one reaction rate and one initial value per region, which it interpolates linearly along the connections; further setters select the time scheme, the mass lumping, the Newton parameters and the output stride. After the setters, `set_coefficients()` validates and stores everything the assembly needs.

7.6 Assembly procedures

`assembly()` loops over the connections and, on each connection, over its cells. The elemental blocks of the piecewise linear basis on a cell of size h are accumulated as triplets and assembled into compressed Eigen sparse matrices: the consistent mass block, its lumped variant when mass lumping is selected, and the stiffness block scaled by the diffusion coefficient of the connection. The nonlinear reaction of the Fisher–Kolmogorov model is integrated by Gauss quadrature applied to the nodal interpolants of the rate and of the state, with a vertex rule consistent with the lumped mass when lumping is on. The Kirchhoff conditions at the vertices need no explicit treatment, since they arise naturally from the weak form when the test functions are continuous across a vertex; Dirichlet data are imposed algebraically on the selected vertices.

7.7 Solvers

The linear heat equation factorises its θ-method matrix once, before the time loop, with the sparse LU of Eigen and reuses the factorisation at every step. The Fisher–Kolmogorov schemes refactorise: the semi-implicit scheme at every step, because the extrapolated reaction changes the matrix, and the fully implicit scheme at every iteration of the damped and safeguarded Newton method described in Section 4. The eigenvalue problems of Section 3 are small enough for a dense generalised symmetric eigensolver. The performance study of Section 5 adds the measured alternatives: the symmetric LDL^T factorisation on the natural ordering, the per-connection static condensation with its dense 83×83 interface and the OpenMP, MPI and hybrid variants of the condensed step.

7.8 Exporting results

`sol_export()` writes ParaView files. Time-dependent problems produce a `solution.pvd` collection referencing one `solution_XXXX.vtp` polyline file per stored step, with a configurable stride to bound the output volume; the spectral problems write the domain and one file per eigenfunction, together with a CSV of the eigenvalues. The drivers add their own CSV summaries, timings, staging times and regional means, which are the inputs of the figures.

7.9 Result postprocessing

Post-processing is Python. Each figure of the report is produced by one script under `scripts/`, reading stored outputs, in almost all cases the results of the corresponding benchmark record; shared modules fix the visual conventions, so the figures of every chapter share one style. The records close the loop. `scripts/run-benchmark.sh` rebuilds the executable of one benchmark, reruns it and refreshes its stored results, and each record's README documents the reading of its experiment; `scripts/verify-figures.py` re-derives from the stored outputs several hundred of the quantities quoted in the report, from mesh sizes and factor fills to staging times and validation differences, and fails when any of them drifts. The figures of Section 5 are generated in the same way from the CSV records of the performance studies.

7.10 Example of code usage

The driver of Test E3 (Section 3.1.3) shows the complete arc of the interface: construction with the physical and time parameters, data supplied as function objects, initialisation from a graph file, coefficient evaluation, assembly, solution and the error check against the exact solution.

```
#include "parabolic_problem.hpp"
#include "star_sine_functions.hpp"
#include <iostream>
#include <memory>
#include <string>

int main(int argc, char *argv[]) {
    const double diffusion = 1.0;
    const double T = 1.0;
    const double deltat = 0.005;
    const double theta = 1.0;

    std::string graph_file = "data/star_4.txt";
    char *local_argv[] = {argv[0], graph_file.data()};

    femg::parabolic_problem problem(diffusion, T, deltat, theta);
    problem.set_initial_condition(std::make_shared<StarSineInitialCondition>());
    problem.set_source_function(std::make_shared<StarSineForcingTerm>());
    problem.set_dirichlet_vertices(
        {0, 1, 2, 3, 4},
        [](std::size_t, double) {
            return 0.0;
        });
    problem.set_output_directory("output/visualization/star_sine");

    if (argc >= 2) {
        problem.init(argc, argv);
    } else {
        problem.init(2, local_argv);
    }

    problem.set_coefficients();
    problem.assemble_matrices();
```

```
    problem.print_matrix_summary(std::cout);
    problem.solve();

    const StarSineExactSolution exact_solution;
    const double error_l2 = problem.compute_l2_error(exact_solution);
    std::cout << "Final L2 error against u = sin(2*pi*x): "
              << error_l2 << "\n";

    return 0;
}
```

Building and running the example requires three commands:

```
cmake -S . -B build
cmake --build build
./build/test_heat_star_sine
```

The solution lands in `output/visualization/star_sine/solution.pvd`, ready for ParaView. The same pattern, a constructor, the setters, `init`, coefficients, assembly, solve and export, is the whole interface a new experiment needs.